Department of Computer Science and Engineering

Assignment Questions

Course Code & Name : DSA0307 – Natural Language Processing

Student Name & Register Number:

Charandeep.N.C (192472196)

A.Sai Rohit (192472144)

D. Sri Anjaneya (192425219)

L. Sanjay Kumar (192425226)

K.Mounith (192425090)

A. Assignment Information

Particular	Details
Department:	CSE/IT/AI&DS/AI&ML
Programme:	CSE
Course Code & Course Name	DSA0307 – Natural Language Processing
Academic Year / Batch	2026
Faculty Name	Dr. K. Anbazhagan
Assignment Title	Design and implement a Python-based NLP application for real-time text processing using tokenization, stemming/lemmatization, POS tagging, and sentiment or text classification
Date of Issue	31/08/2026
Date of Submission	02/09/26
Maximum Marks	100
Course Outcome(s) – CO	CO2 - To design an innovative application using NLP components

Bloom's Taxonomy Level	L3 – Apply, L5 – Evaluate
SDG Mapping (SDG 1-17)	SDG8 - Decent Work and Economic Growth & SDG9 - Industry, Innovation and Infrastructure
Industry / Societal Relevance	

B. Assignment Problem / Challenge

Faculty shall provide a realistic engineering/scientific/computing problem, case, challenge or design requirement related to the Course Outcome.

The assignment should preferably require students to:

- Apply course concepts rather than reproduce theory.
- Identify and analyze the problem.
- Consider requirements and constraints.
- Develop or compare alternative solutions where appropriate.
- Use relevant modern engineering/computing/design tools.
- Interpret results.
- Make and justify engineering decisions.
- Consider appropriate technical, economic, environmental, societal, ethical, safety or sustainability factors wherever relevant.

C. Problem Statement

Problem / Case / Design Challenge:

Design and implement a Python-based NLP application for real-time text processing using tokenization, stemming/lemmatization, POS tagging, and sentiment or text classification

a) Explain the role of **tokenization, stemming/lemmatization, POS tagging, and sentiment or text classification** in NLP. Design a suitable **text-processing workflow** for converting raw user input into meaningful linguistic information. Represent the workflow using a suitable **flow diagram or pseudocode**.

b) Implement a **Python-based NLP preprocessing module** that accepts a text sentence as input and performs **tokenization, stop-word removal, stemming/lemmatization, and POS tagging**. Display the output generated at each processing stage and explain the NLP technique used.

c) Develop a **Python-based sentiment or text classification module** using the processed text. Extract suitable features such as **word frequency, n-grams, or TF-IDF**, apply an appropriate classification algorithm, and predict the sentiment/class of a new input sentence. Provide the **Python code and sample input/output**.

d) Test the developed application using multiple text inputs and evaluate its performance using suitable measures such as **accuracy, precision, recall, and F1-score**. Discuss the effect of

preprocessing techniques on classification performance and identify the limitations of the implemented NLP approach.

D. Requirements and Constraints

Students shall identify or be provided with relevant requirements and constraints. Examples include: •

Functional requirements

- Performance requirements
- Technical constraints
- Cost, Time
- Resources
- Safety
- Reliability
- Sustainability
- Environmental and Ethical considerations
- Standards/codes
- User requirements
- Manufacturability
- Power/energy
- Security/privacy

Only constraints relevant to the particular course/problem need to be considered.

E. Student Work

1. Problem Understanding and Formulation

Explain:

- What is the problem?
- What are the expected outcomes?
- What information/data is available?
- What assumptions are made?
- What constraints must be satisfied?

2. Application of Course Knowledge

Identify and apply the relevant:

- Principles
- Theories
- Mathematical models
- Algorithms
- Engineering concepts
- Scientific concepts
- Design methodologies

Show necessary calculations, derivations or reasoning.

3. Solution / Design / Methodology

Develop the proposed solution. Depending upon the nature of the course, this may include:

- Design
- Algorithm
- Model
- Circuit
- Architecture
- Experiment
- Process
- Prototype
- Simulation
- Case analysis
- Data analysis
- Mathematical solution

Where appropriate, consider more than one possible solution.

4. Use of Modern Tools

Use appropriate tools wherever relevant.

Examples:

CSE/IT: Python, MATLAB, TensorFlow, cloud platforms, Git, simulation tools

ECE/EEE: MATLAB/Simulink, Cadence, Synopsys, Vivado, Multisim, LTspice

Mechanical: ANSYS, SolidWorks, CATIA, MATLAB

Civil: STAAD.Pro, ETABS, AutoCAD, Revit

Biomedical: MATLAB, LabVIEW, signal/image-processing tools

Biotechnology: Bioinformatics/data-analysis/simulation tools

Students shall provide appropriate evidence of tool usage and outputs.

5. Results and Validation

Present the results using appropriate:

- Tables
- Graphs
- Simulation outputs
- Experimental observations
- Screenshots
- Performance metrics
- Statistical analysis
- Test results

Validate whether the proposed solution satisfies the stated requirements.

6. Analysis and Engineering Decision

Interpret the results.

Where applicable:

- Compare alternatives.
- Identify advantages and limitations.
- Discuss trade-offs.

- Explain why the final solution was selected.
- Justify the decision using quantitative or qualitative evidence.

7. Broader Considerations

Where relevant to the problem, discuss the impact of the proposed solution with respect to:

- Sustainability
- Environment
- Society
- Safety
- Ethics
- Economics
- Accessibility
- Professional responsibility

8. Conclusion

Summarize:

- Proposed solution
- Major findings
- Achievement of requirements
- Limitations
- Possible improvements

9. Student Reflection

- What did you learn from this assignment beyond what was directly taught in the classroom?
- If you were given additional time/resources, what would you improve and why?

10. References

Use an appropriate citation format and acknowledge all external sources, datasets, standards, software and AI-assisted tools used.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5

TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10
Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5
Documentation & reflection			L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is not required to map to every PO.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

_____ **Areas requiring improvement:**

_____ **Corrective / Improvement Action:**

_____ **Action to be implemented in the next offering:**

_____

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action

1. Problem Understanding and Formulation:

1.1 Problem Statement

Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to process and understand human language.

The objective of this assignment is to design and implement a Python-based NLP application that accepts a sentence from the user and performs the following operations:

- Tokenization
- Stop-word removal
- Stemming/Lemmatization
- POS tagging
- Feature extraction using TF-IDF
- Sentiment classification
- Performance evaluation

The system processes raw text and converts it into meaningful linguistic information before predicting whether the sentence is **positive or negative**.

1.2 Expected Outcomes The

developed application should:

- Accept text from the user.
- Divide text into individual tokens.

- Remove unnecessary stop words.
- Convert words into their root/base forms.
- Identify the grammatical category of words.
- Extract numerical features using TF-IDF.
- Train a sentiment classification model.
- Predict the sentiment of new input.
- Evaluate the model using accuracy, precision, recall, and F1-score.

1.3 Available Data

A small labelled dataset is used for demonstration.

Sentence	**Sentiment**
I love this product	Positive
The movie was excellent	Positive
This phone works very well	Positive
I enjoyed the service	Positive
The food was delicious	Positive
I hate this product	Negative
The movie was boring	Negative
This phone is very bad	Negative
I disliked the service	Negative

The food was terrible	Negative

1.4 Assumptions

- The input is in English.
- The input contains normal words and sentences.
- The dataset contains labelled positive and negative examples.
- The classification problem is binary.
- The application is mainly designed for educational purposes.

2. Application of Course Knowledge:

2.1 Tokenization

Tokenization is the process of dividing text into smaller units called **tokens**.

Example:

Input:

I really enjoyed this movie.

Tokens:

I | really | enjoyed | this | movie

Tokenization is generally the first step in text processing.

2.2 Stop-Word Removal

Stop words are frequently occurring words that usually provide limited information for classification. Examples include:

the, is, a, an, this, and, of

Example:

This is a good movie. After

stop-word removal:

good movie

2.3 Stemming

Stemming reduces words to their approximate root form.

Example:

Word	Stem
playing	play
played	play
Word	**Stem**
connected	connect
studies	studi

Stemming is fast but may sometimes produce words that are not valid dictionary words.

2.4 Lemmatization

Lemmatization converts words into their meaningful base or dictionary form.

Example:

Word	Lemma
running	run

studies	study
cars	car
better	good

Lemmatization generally produces more meaningful results than stemming.

2.5 POS Tagging

POS stands for **Part-of-Speech**.

It assigns a grammatical category to each word.

For example:

The beautiful girl sings.

Possible POS tags:

The/DT beautiful/JJ girl/NN sings/VBZ Here:

- DT = Determiner
- JJ = Adjective
- NN = Noun
- VBZ = Verb

2.6 TF-IDF

TF-IDF stands for **Term Frequency-Inverse Document Frequency**.

It converts text into numerical features that can be processed by machine-learning algorithms.

The formula is:

$$TF - IDF(t, d) = TF(t, d) \times IDF(t)$$

where:

- TF = Term Frequency
- IDF = Inverse Document Frequency
- t = term
- d = document

2.7 Sentiment Classification

Sentiment classification identifies the opinion expressed in text.

In this assignment:

Positive → 1

Negative → 0

Logistic Regression is used as the classification algorithm because it is simple, fast, and suitable for binary classification.

2.8 NLP Workflow

START

|

Enter User Text

|

Tokenization

|

Stop-word Removal

|

Stemming/Lemmatization

|

POS Tagging

|

TF-IDF Extraction

|

v

Logistic Regression

|

Sentiment Prediction

|

Positive / Negative

|

END

2.9 Pseudocode

```
BEGIN

Load NLP libraries Create labelled dataset

FOR each sentence in dataset:

    Convert sentence to lowercase

    Tokenize sentence
```

Remove stop words

Perform stemming

END FOR

Create TF-IDF features

Split data into training and testing sets

Train Logistic Regression model

INPUT a new sentence

Preprocess the input sentence

Convert input into TF-IDF features

Predict sentiment

IF prediction is 1:

Display Positive ELSE:

Display Negative

Calculate accuracy

Calculate precision

Calculate recall

Calculate F1-score

END

3. Solution / Design / Methodology:

3.1 Proposed Method

The proposed application consists of the following stages:

1. **Input:** User enters a sentence.

2. **Preprocessing:** Text is converted to lowercase and tokenized.
3. **Stop-word removal:** Unnecessary words are removed.

4. **Stemming:** Words are reduced to root forms.

5. **POS tagging:** Grammatical categories are identified.

6. **Feature extraction:** TF-IDF converts text into numerical vectors.

7. **Classification:** Logistic Regression predicts the sentiment.

8. **Output:** Positive or Negative sentiment is displayed.

3.2 Python Preprocessing Program

```
import nltk from nltk.tokenize import

word_tokenize from nltk.corpus import

stopwords from nltk.stem import PorterStemmer

from nltk import pos_tag nltk.download('punkt')

nltk.download('punkt_tab')

nltk.download('stopwords')

nltk.download('averaged_perceptron_tagger_eng')

# Input text = input("Enter a

sentence: ")

# Convert to lowercase text

= text.lower() # Tokenization

tokens = word_tokenize(text)

print("\n1. Tokens:")

print(tokens)

# Stop-word removal
```

```
stop_words = set(stopwords.words("english"))

filtered_words = [     word for word in tokens

if word.isalpha() and word not in stop_words

]

print("\n2. After Stop-word Removal:")

print(filtered_words) # Stemming

stemmer = PorterStemmer()

stemmed_words = [

stemmer.stem(word)     for word in

filtered_words

]

print("\n3. After Stemming:")
print(stemmed_words) # POS
Tagging pos_tags =
pos_tag(filtered_words) print("\n4.
POS Tags:") print(pos_tags)
```

Sample Input

I really enjoyed this beautiful movie

Sample Output

1. Tokens:

['i', 'really', 'enjoyed', 'this', 'beautiful', 'movie']

2. After Stop-word Removal:

['really', 'enjoyed', 'beautiful', 'movie']

3. After Stemming:

['realli', 'enjoy', 'beauti', 'movi']

4. POS Tags:

[('really', 'RB'),

('enjoyed', 'VBD'),

('beautiful', 'JJ'),

('movie', 'NN')]

3.3 Sentiment Classification Code

```
import nltk



from nltk.corpus import stopwords from

nltk.stem import PorterStemmer from

nltk.tokenize import word_tokenize



from sklearn.feature_extraction.text import
TfidfVectorizer from sklearn.linear_model import
LogisticRegression from sklearn.model_selection import
train_test_split from sklearn.metrics import
accuracy_score from sklearn.metrics import
precision_score from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
```

```
nltk.download('punkt')

nltk.download('punkt_tab')

nltk.download('stopwords')


# Dataset

texts = [

    "I love this product",

    "The movie was excellent",

    "This phone works very well",

    "I enjoyed the service",

    "The food was delicious",

    "This product is amazing",

    "I really liked the movie",

    "The service was wonderful",


    "I hate this product",

    "The movie was boring",

    "This phone is very bad",

    "I disliked the service",
```

```
    "The food was terrible",
    "This product is awful",
    "I hated the movie",
    "The service was poor"
]

labels = [
    1, 1, 1, 1, 1, 1, 1, 1,
    0, 0, 0, 0, 0, 0, 0, 0
]

stop_words = set(stopwords.words("english"))
stemmer = PorterStemmer()

def preprocess(text):

    text = text.lower()

    tokens = word_tokenize(text)
```

```
    tokens = [          word for word in tokens          if
word.isalpha() and word not in stop_words
    ]


    tokens = [
stemmer.stem(word)
for word in tokens
    ]

    return " ".join(tokens)



# Preprocess dataset
processed_texts = [
preprocess(text)
for text in texts
]
```

```
 # TF-IDF vectorizer =
 TfidfVectorizer()
```

```
X = vectorizer.fit_transform(processed_texts)
y = labels

# Train-test split
X_train, X_test, y_train, y_test =
train_test_split( X, y,
    test_size=0.25,
random_state=42,
stratify=y
)

# Train model
model = LogisticRegression()
model.fit(X_train, y_train)

# Prediction y_pred =
model.predict(X_test)
```

```
# Evaluation accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(y_test, y_pred,

zero_division=0) recall = recall_score(y_test, y_pred,

zero_division=0) f1 = f1_score(y_test, y_pred,

zero_division=0)


print("\nEvaluation Results") print("--

----------") print("Accuracy :",

accuracy) print("Precision:",

precision) print("Recall   :", recall)

print("F1-score :", f1)


# New input

user_text = input(

    "\nEnter a sentence for sentiment prediction: "

)


processed_input = preprocess(user_text)
```

```
input_vector = vectorizer.transform(
    [processed_input]
)

prediction = model.predict(input_vector)[0]

print("\nOriginal Text:")
print(user_text)

print("\nProcessed Text:") print(processed_input)

if prediction == 1:
    print("\nPredicted Sentiment: POSITIVE")
else:
    print("\nPredicted Sentiment: NEGATIVE")
```

Sample Input

I really love this amazing product

Sample Output Original Text: I

really love this amazing product

Processed Text: realli love amaz

product

Predicted Sentiment: POSITIVE

Another example:

Enter a sentence for sentiment prediction:

This product is very bad

Original Text: This

product is very bad

Processed Text:

product bad

Predicted Sentiment: NEGATIVE

4. Use of Modern Tools

The following modern tools and technologies are used:

Tool/Technology	**Purpose**
Python	Main programming language
NLTK	NLP preprocessing
Scikit-learn	Machine learning
TF-IDF	Feature extraction
Logistic Regression	Sentiment classification
Jupyter Notebook/Google Colab/VS Code	Program execution

NLTK

NLTK is used for:

- Tokenization
- Stop-word removal
- Stemming
- POS tagging

Scikit-learn

Scikit-learn is used for:

- TF-IDF
- Dataset splitting
- Logistic Regression
- Performance evaluation

5. Results and Validation:

The application was tested using multiple positive and negative sentences.

Input Sentence	Expected Sentiment
I love this phone	Positive
This movie is excellent	Positive
The food was delicious	Positive
This service is amazing	Positive
I hate this product	Negative

The movie was boring	Negative
This phone is terrible	Negative
The service was poor	Negative

The application successfully performs:

Requirement	**Result**
Text input	Successful
Requirement	**Result**
Tokenization	Successful
Stop-word removal	Successful
Stemming	Successful
POS tagging	Successful
TF-IDF extraction	Successful
Sentiment classification	Successful
Performance evaluation	Successful

Evaluation Measures

Accuracy:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision:

$$Precision = \frac{TP}{TP + FP}$$

Recall:

$$Recall = \frac{TP}{TP + FN}$$

F1-score:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

The exact metric values should be taken from the output produced when the program is executed because the train-test split can affect the results.

6. Analysis and Engineering Decision:

The developed system combines traditional NLP techniques with machine learning.

TF-IDF was selected for feature extraction because it is simple and effective for representing the importance of words in documents.

Logistic Regression was selected because:

- It is suitable for binary classification.
- It is computationally efficient.
- It works well with TF-IDF features.
- It is easy to implement.
- It requires relatively low computational resources.

Effect of Preprocessing

Without preprocessing, text contains unnecessary words, punctuation, and different word forms.

For example:

I am REALLY enjoying this movie!

After preprocessing:

realli enjoy movi

This reduces unnecessary information and can improve the efficiency of classification.

However, excessive preprocessing can sometimes remove useful information. For example, removing words related to negation may affect the meaning of a sentence.

Limitations

The system has the following limitations:

- Small training dataset.
- Limited vocabulary.
- Difficulty understanding sarcasm.
- Difficulty handling complex sentences.
- Limited understanding of context.
- Stemming can generate non-dictionary words.
- Primarily designed for English text.

A larger dataset and advanced NLP models could improve the performance.

7. Broader Considerations:

Sustainability

The proposed system uses TF-IDF and Logistic Regression, which require relatively low computational resources compared with large deep-learning models.

Therefore, it is suitable for lightweight NLP applications.

Society

The system can be used for:

- Customer review analysis
- Product feedback
- Social media analysis
- Service evaluation
- Survey analysis

Ethics

NLP applications should consider:

- User privacy
- Data security
- Bias in datasets
- Incorrect predictions
- Responsible use of user-generated text

Personal and confidential information should be processed only with appropriate permission.

Economic Impact

Automated sentiment analysis can reduce the manual effort required to analyze large numbers of customer reviews and feedback.

The application is relevant to:

- **SDG 8 – Decent Work and Economic Growth**
- **SDG 9 – Industry, Innovation and Infrastructure**

8. Conclusion:

- The Python-based NLP application was successfully designed and implemented for real-time text processing.
- The system performs tokenization, stop-word removal, stemming, POS tagging, TFIDF feature extraction, and sentiment classification.
- TF-IDF converts processed text into numerical features, while Logistic Regression predicts whether a sentence is positive or negative. The system can also be evaluated using accuracy, precision, recall, and F1-score. ○ The assignment demonstrates how different NLP techniques can be combined to build a complete text-processing and classification system.
- The major limitation is the small demonstration dataset. Future improvements can include using a larger real-world dataset, lemmatization, n-grams, hyperparameter tuning, better negation handling, and advanced transformer-based models.

9. Student Reflection:

What did I learn from this assignment beyond what was taught in the classroom?

I learned how different NLP techniques can be combined to create a complete NLP application. I understood the practical use of tokenization, stop-word removal, stemming, POS tagging, and TF-IDF. I also learned how machine-learning algorithms can be used to classify text and how evaluation metrics are used to measure model performance. This assignment also helped me understand the importance of preprocessing because the quality of processed text can directly affect classification performance.

What would I improve with additional time and resources?

If I had additional time and resources, I would use a larger real-world dataset and compare different machine-learning algorithms such as Naive Bayes, SVM, and Logistic Regression.

10. References:

1. Bird, S., Klein, E., & Loper, E. **Natural Language Processing with Python**. O'Reilly Media.
2. Manning, C. D., Raghavan, P., & Schütze, H. **Introduction to Information Retrieval**.

Cambridge University Press.

3. **NLTK Documentation – Natural Language Toolkit**, Python NLP library.
4. **Scikit-learn Documentation – Machine Learning in Python**.
5. **Python Documentation – Python Software Foundation**.